DISCRETE MATHEMATICS

CSE 2213 • Compact Study Notes Based on the UIU Summer 2026 Lecture Outline

1. Introduction and Propositional Logic

Discrete mathematics studies distinct, countable structures used in computer science. A proposition is a declarative sentence that is either true (T) or false (F), but not both. Let p and q be propositions.

Connective	Meaning	True when…
¬p	NOT p	p is false
p ∧ q	p AND q	both p and q are true
p ∨ q	p OR q (inclusive)	at least one is true
p → q	if p, then q	false only when p=T and q=F
p ↔ q	p iff q	p and q have the same truth value
p ⊕ q	exclusive OR	exactly one of p, q is true

Truth table for implication and equivalence

p	q	p → q	p ↔ q
T	T	T	T
T	F	F	F
F	T	T	F
F	F	T	T

Key terms: A tautology is always true; a contradiction is always false; a contingency is sometimes true and sometimes false. Two propositions are logically equivalent if they have the same truth values in every case.

Useful logical equivalences

$$\neg(p \wedge q) \equiv (\neg p \vee \neg q) \quad \bullet \quad \neg(p \vee q) \equiv (\neg p \wedge \neg q)$$

$$p \to q \equiv \neg p \vee q \quad \bullet \quad p \leftrightarrow q \equiv (p \to q) \wedge (q \to p)$$

Use truth tables to test validity/equivalence, and equivalence laws to simplify expressions without rebuilding the whole table.

2. Predicates, Quantifiers and Nested Quantifiers

A predicate P(x) becomes a proposition when a value is assigned to x. The domain is the set of allowed values. Quantifiers describe how many domain elements satisfy a predicate.

$$\forall x\, P(x)\text{: “for every x, P(x)”} \qquad \exists x\, P(x)\text{: “there exists an x such that P(x)”}$$

$$\neg\forall x\, P(x) \equiv \exists x\, \neg P(x) \quad \bullet \quad \neg\exists x\, P(x) \equiv \forall x\, \neg P(x)$$

To negate a quantified statement, switch ∀ and ∃ and negate the predicate. With nested quantifiers, order matters: ∀x∃y P(x,y) says each x may have its own y; ∃y∀x P(x,y) says one fixed y works for every x. The second is generally stronger.

3. Basic Proof Techniques

Direct proof: assume the hypothesis and derive the conclusion. Contrapositive: prove ¬q → ¬p instead of p → q. Contradiction: assume the claim is false and derive an impossibility. Proof by cases: split into exhaustive cases. Counterexample: one example disproves a universal claim. Biconditional: prove both directions.

Example (direct proof): If n is even, then n^2 is even. Write $n = 2k$ for an integer k. Then $n^2 = 4k^2 = 2(2k^2)$, so n^2 is even.

Proof discipline: state assumptions, justify each inference, and prove exactly the statement requested. A few confirming examples do not prove a universal statement.

4. Sets and Set Operations

A set is a collection of distinct elements. Notation: $x \in A$ means x belongs to A; $A \subseteq B$ means every element of A is in B; $\emptyset$ is the empty set; U is the universal set; $|A|$ is cardinality; $P(A)$ is the power set.

Operation	Definition / meaning
Union	$A \cup B$ = elements in A or B (or both)
Intersection	$A \cap B$ = elements common to A and B
Difference	$A - B$ = elements in A but not B
Complement	A^c = elements in U that are not in A
Cartesian product	$A \times B$ = ordered pairs (a,b), where $a \in A$ and $b \in B$

$$|A \cup B| = |A| + |B| - |A \cap B|$$

De Morgan’s laws: $(A \cup B)^c = A^c \cap B^c$ and $(A \cap B)^c = A^c \cup B^c$. If A has n elements, then $|P(A)| = 2^n$. For finite sets, $|A \times B| = |A||B|$.

5. Functions

A function $f: A \to B$ assigns every input in A exactly one output in B. A is the domain and B is the codomain; the range is the set of outputs actually reached. Injective (one-to-one): different inputs have different outputs. Surjective (onto): every element of B is reached. Bijective: both injective and surjective, so an inverse function exists.

$$\text{Injective: } f(a)=f(b) \Rightarrow a=b \quad \bullet \quad (f \circ g)(x)=f(g(x))$$

The composition $f \circ g$ means apply g first, then f. A function’s inverse reverses input and output and exists as a function when the original mapping is bijective.

6. Mathematical Induction

Induction proves a statement P(n) for every integer $n \geq n_0$. First prove the base case $P(n_0)$. Then assume P(k) is true and use that assumption to prove P(k+1). Conclude that P(n) holds for all $n \geq n_0$.

Base case: $P(n_0)$ • Inductive step: $P(k) \Rightarrow P(k+1)$

Example pattern: to prove $1+2+\dots+n = n(n+1)/2$, verify $n=1$. Assume the formula for k; then add $k+1$ and simplify: $k(k+1)/2 + (k+1) = (k+1)(k+2)/2$.

7. Counting: Basics, Pigeonhole Principle, Permutations and Combinations

Product rule: if a task has m choices followed by n choices, there are mn outcomes. Sum rule: for mutually exclusive alternatives with m and n outcomes, there are $m+n$ outcomes. Factorial: $n! = n(n-1)\dots2\cdot1$, with $0! = 1$.

Permutations (order matters): $P(n,r) = n!/(n-r)!$

Combinations (order does not matter): $C(n,r) = n!/[r!(n-r)!]$

Permutations with repeated objects: $n!/(n_1!\, n_2!\, \dots\, n_k!)$

Use permutations when different arrangements count as different outcomes; use combinations when only the selected group matters. For independent choices, multiply the number of options. If repetitions are allowed, check whether each position has the same number of choices.

Pigeonhole principle: if more than n objects are placed in n boxes, at least one box contains at least two objects. General form: placing N objects into k boxes means some box has at least $\lceil N/k \rceil$ objects.

8. Recurrence Relations

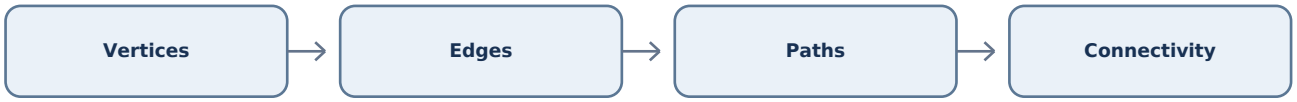
A recurrence defines a sequence using earlier terms, together with initial conditions. Example: $a_n = a_{n-1} + 2$ with $a_0 = 1$ gives 1, 3, 5, 7, … . A recurrence may model recursive algorithms and counting processes.

Arithmetic pattern: $a_n = a_{n-1} + d \Rightarrow a_n = a_0 + nd$

To solve a recurrence, identify its type, use initial values, and derive a closed form or a growth pattern where possible. The outline names recurrence relations but does not specify a particular solving method.

9. Graphs and Graph Models

A graph $G=(V,E)$ consists of vertices V and edges E. It models connections such as computer networks, routes, and relationships. In an undirected graph, edges have no direction; in a directed graph, each edge has a direction. A simple graph has no loops or multiple edges.



Term	Meaning
Degree	Number of incident edges; a loop contributes 2 in an undirected graph
Walk / trail / path	A walk may repeat vertices and edges; a trail does not repeat edges; a path does not repeat vertices
Cycle	A closed path with no repeated vertices except the start/end
Connected graph	Every pair of vertices is joined by a path
Complete graph K_n	Every pair of distinct vertices is connected
Bipartite graph	Vertices can be split into two sets; each edge joins different sets

Handshaking theorem (undirected graph): $\Sigma \deg(v) = 2|E|$

A graph model is selected by deciding what the vertices and edges represent, and whether direction or multiple edges are needed. In a directed graph, distinguish in-degree from out-degree.

10. Graph Representation and Isomorphism

An adjacency list stores the neighbors of each vertex and is often space-efficient for sparse graphs. An adjacency matrix uses an $n \times n$ matrix whose entry indicates whether an edge exists (or stores its weight). An incidence matrix relates vertices to edges.

Two graphs are isomorphic if a one-to-one correspondence between their vertices preserves adjacency. Compare vertex count, edge count, degree sequence, and cycles as quick checks; matching these properties alone does not always prove isomorphism.

11. Paths and Graph Connectivity

A path connects vertices; its length is the number of edges used. A graph is connected if every pair of vertices has a path between them. A disconnected graph splits into connected components. A bridge is an edge whose removal increases the number of components; a cut vertex is a vertex whose removal increases the number of components.

Breadth-first search (BFS) explores level by level using a queue; depth-first search (DFS) explores deeply before backtracking, often using recursion or a stack. In an unweighted graph, BFS finds shortest paths in number of edges.

12. Trees and Their Properties

A tree is a connected, undirected graph with no cycles. For a tree with n vertices, the number of edges is $n-1$. There is exactly one simple path between every pair of vertices. Removing any edge disconnects a tree; adding one new edge creates a cycle.

Tree: $|E| = |V| - 1$

A rooted tree designates one vertex as the root. Parent, child, sibling, ancestor, descendant, leaf, and internal vertex describe relationships. The height is the maximum distance from the root to a leaf (conventions can vary). A spanning tree of a connected graph includes every vertex and is a tree subgraph.

13. Binary Search Trees and Tree Traversal

A binary tree has at most two children per node. In a binary search tree (BST), values in the left subtree are smaller than the node and values in the right subtree are larger, assuming distinct keys and this convention. Search, insertion, and deletion follow these comparisons.

Traversal	Visit order	Use / result
Preorder	Root, Left, Right	Root is processed before its subtrees
Inorder	Left, Root, Right	Produces sorted order for a BST
Postorder	Left, Right, Root	Children are processed before the root
Level order	Level by level	Usually implemented with a queue

A BST's operation cost depends on its height h: typical search/insert is O(h). A balanced tree has height around $\log_2 n$; a highly skewed tree can have height close to n.

14. Applications of Trees

Trees represent file systems, organizational hierarchies, expression syntax, decision processes, and search structures. Expression trees store operators in internal nodes and operands in leaves. Spanning trees connect all vertices without cycles; they are useful when a network needs connectivity without redundant links.

Quick Revision

Topic	Remember
Logic	Implication is false only for T → F; negate quantifiers by switching ∀/∃ and negating the predicate.
Sets / functions	Union = either; intersection = both. Bijective functions have inverses.
Induction	Base case + inductive step.
Counting	Multiply sequential choices; permutations care about order, combinations do not.
Graphs	$\Sigma \deg(v) = 2\|E\|$ for undirected graphs; BFS finds unweighted shortest paths.
Trees	Connected + acyclic; n vertices means $n-1$ edges.
BST traversals	Inorder gives sorted keys under the standard BST ordering.

Source alignment: These notes follow the topics listed in the supplied CSE 2213 Discrete Mathematics lecture outline: propositional logic; predicates and quantifiers; basic proof techniques; sets; functions; induction; counting; pigeonhole principle; permutations and combinations; recurrence relations; graphs and graph models; graph types, representation and isomorphism; paths and connectivity; trees, BSTs, traversal, and applications. Explanations are concise study aids, not a replacement for the textbook or instructor's full lecture material.